

Unfold#

<https://pytorch.org/docs/stable/generated/torch.nn.Unfold.html>

Description:

Extracts sliding local blocks from a batched input tensor. Consider a batched input tensor of shape $(N, C, *)$, where N is the batch dimension, C is the channel dimension, and $*$ represent arbitrary spatial dimensions. This operation flattens each sliding kernel_size -sized block within the spatial dimensions of input into a column (i.e., last dimension) of a 3-D output tensor of shape $(N, C \times \prod(\text{kernel_size}), L)$, where $C \times \prod(\text{kernel_size})$ is the total number of values within each block (a block has $\prod(\text{kernel_size})$ spatial locations each containing a C -channeled vector), and L is the total number of such blocks: where spatial_size is formed by the spatial dimensions of input ($*$ above), and ddd is over all spatial dimensions. Therefore, indexing output at the last dimension (column dimension) gives all values within a certain block. The padding, stride and dilation arguments specify how the sliding blocks are retrieved.

Function Signature

```
class torch.nn.Unfold(kernel_size, dilation=1, padding=0,
stride=1)[source]#
```

Parameters

Shape:

```
extra_repr()[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> fold_params = dict(kernel_size=..., dilation=...,
padding=..., stride=...)
>>> fold = nn.Fold(output_size=..., **fold_params)
>>> unfold = nn.Unfold(**fold_params)
```

```
fold(unfold(input)) == divisor * input
```

```
>>> input_ones = torch.ones(input.shape, dtype=input.dtype)
>>> divisor = fold(unfold(input_ones))
```

```
>>> unfold = nn.Unfold(kernel_size=(2, 3))
>>> input = torch.randn(2, 5, 3, 4)
>>> output = unfold(input)
>>> # each patch contains 30 values (2x3=6 vectors, each of 5
channels)
>>> # 4 blocks (2x3 kernels) in total in the 3x4 input
>>> output.size()
torch.Size([2, 30, 4])

>>> # Convolution is equivalent with Unfold + Matrix
Multiplication + Fold (or view to output shape)
>>> inp = torch.randn(1, 3, 10, 12)
>>> w = torch.randn(2, 3, 4, 5)
>>> inp_unf = torch.nn.functional.unfold(inp, (4, 5))
>>> out_unf = inp_unf.transpose(1, 2).matmul(w.view(w.size(0),
-1).t()).transpose(1, 2)
>>> out = torch.nn.functional.fold(out_unf, (7, 8), (1, 1))
>>> # or equivalently (and avoiding a copy),
>>> # out = out_unf.view(1, 2, 7, 8)
>>> (torch.nn.functional.conv2d(inp, w) - out).abs().max()
tensor(1.9073e-06)
```

Notes & Warnings

NOTE

Note Fold calculates each combined value in the resulting large tensor by summing all values from all containing blocks. Unfold extracts the values in the local blocks by copying from the large tensor. So, if the blocks overlap, they are not inverses of each other. In general, folding and unfolding operations are related as follows. Consider Fold and Unfold instances created with the same parameters: `>>> fold_params = dict(kernel_size=..., dilation=..., padding=..., stride=...) >>> fold = nn.Fold(output_size=..., **fold_params) >>> unfold = nn.Unfold(**fold_params)` Then for any (supported) input tensor the following equality holds: `fold(unfold(input)) == divisor * input` where divisor is a tensor that depends only on the shape and dtype of the input: `>>> input_ones = torch.ones(input.sh`

⚠ WARNING

Warning Currently, only 4-D input tensors (batched image-like tensors) are supported.

Detailed Documentation

Documentation

Extracts sliding local blocks from a batched input tensor.

Consider a batched input tensor of shape $(N, C, *, *)$ $(N, C, *)$ $(N, C, *)$, where NNN is the batch dimension, CCC is the channel dimension, and $***$ represent arbitrary spatial

dimensions. This operation flattens each sliding `kernel_size`-sized block within the spatial dimensions of input into a column (i.e., last dimension) of a 3-D output tensor of shape $(N, C \times \prod(\text{kernel_size}), L)$ ($N, C \times \prod(\text{kernel_size}), L$), where $C \times \prod(\text{kernel_size})$ is the total number of values within each block (a block has $\prod(\text{kernel_size}) \prod(\text{kernel_size})$ spatial locations each containing a `CCC`-channeled vector), and `LLL` is the total number of such blocks:

where `spatial_size` is formed by the spatial dimensions of input (`***` above), and `ddd` is over all spatial dimensions.

Therefore, indexing output at the last dimension (column dimension) gives all values within a certain block.

The padding, stride and dilation arguments specify how the sliding blocks are retrieved.

`stride` controls the stride for the sliding blocks.

`padding` controls the amount of implicit zero-paddings on both sides for padding number of points for each dimension before reshaping.

`dilation` controls the spacing between the kernel points; also known as the *à trous* algorithm. It is harder to describe, but this link has a nice visualization of what dilation does.

`kernel_size` (int or tuple) – the size of the sliding blocks

`dilation` (int or tuple, optional) – a parameter that controls the stride of elements within the neighborhood. Default: 1

`padding` (int or tuple, optional) – implicit zero padding to be added on both sides of

input. Default: 0

stride (int or tuple, optional) – the stride of the sliding blocks in the input spatial dimensions. Default: 1

If kernel_size, dilation, padding or stride is an int or a tuple of length 1, their values will be replicated across all spatial dimensions.

For the case of two input spatial dimensions this operation is sometimes called im2col.

Fold calculates each combined value in the resulting large tensor by summing all values from all containing blocks. Unfold extracts the values in the local blocks by copying from the large tensor. So, if the blocks overlap, they are not inverses of each other.

In general, folding and unfolding operations are related as follows. Consider Fold and Unfold instances created with the same parameters:

Then for any (supported) input tensor the following equality holds:

where divisor is a tensor that depends only on the shape and dtype of the input:

When the divisor tensor contains no zero elements, then fold and unfold operations are inverses of each other (up to constant divisor).

Currently, only 4-D input tensors (batched image-like tensors) are supported.

Input: $(N, C, *, *)$ $(N, C, *)$ $(N, C, *)$

Output: $(N, C \times \prod(\text{kernel_size}), L)$ $(N, C \times \prod(\text{kernel_size}), L)$
 $(N, C \times \prod(\text{kernel_size}), L)$ as described above

Return the extra representation of the module.

Runs the forward pass.